

Knowledge graph completion based on a hierarchical graph attention network with structural information

Jie Xu ^{1,*}, Shumao Zhang ¹, Haodiao Xie , Hailing Zhang , Ke Miao , Qiuru Fu

University of Electronic Science and Technology of China, Xiyuan Road 2006th, Chengdu, 611731, Sichuan, China

ARTICLE INFO

Keywords:

Knowledge graph
Product design and manufacturing systems
Graph neural network
Hierarchical attention mechanism

ABSTRACT

Recently, knowledge graphs have played a crucial role in enhancing the cognitive capabilities of product design and manufacturing systems (PDMS). However, most knowledge graphs are neither complete nor comprehensive, which often leads to suboptimal decisions in PDMS. Much of the existing research on knowledge graph completion has overlooked the fact that knowledge graphs are heterogeneous, containing multiple types of relations. To address this issue, we introduce a hierarchical graph attention network that facilitates learning from heterogeneous graphs. The proposed model incorporates a dual-level attention mechanism—entity-level and relation-level attention. This hierarchical approach captures the importance of both entities and relations, enabling more fine-grained feature extraction and improving the model's ability to learn from key parameters and complex relationships within PDMS. Furthermore, we incorporate centrality encoding and relation encoding into the graph learning process, which enriches the structural representation with valuable global information. Finally, comprehensive experiments demonstrate the superiority of our proposed model over several state-of-the-art approaches, showing significant improvements on benchmark datasets.

1. Introduction

Current product design and manufacturing systems (PDMS) have embraced perception technologies with excellent performance. By enhancing the cognitive capabilities of PDMS, these systems can more accurately perceive changes in the external environment, optimize data processing, and improve decision-making efficiency, thereby driving more intelligent technological applications and industrial development [1].

Knowledge graphs (KGs) play a crucial role in enhancing the cognitive capabilities of PDMS. Information in knowledge graphs is commonly represented as triples (h, r, t) , where h and t denote the head and tail entities, respectively, and r indicates the relation connecting them. Thus, knowledge graphs can model a vast amount of knowledge. By structuring and linking domain-specific information, KGs empower PDMS to process complex design, production, and operational knowledge, thereby enhancing their ability to understand and reason about various elements in the design and manufacturing processes [2].

Despite the success of KGs in PDMS, most real-world knowledge graphs suffer from incompleteness and noisy information [3]. In recent years, many studies have focused on the task of knowledge graph com-

pletion (KGC) [4,5]. Specifically, KGC aims to predict the head entity in a query $(?, r, t)$ or the tail entity in a query $(h, r, ?)$.

Recent advancements in KGC research primarily focus on knowledge graph embedding techniques. These models aim to learn latent, low-dimensional representations of entities [6,7] and relations [8] that capture their semantic meanings. By learning such embeddings, it becomes possible to preserve the structural information inherent in knowledge graphs. The model then predicts missing entities using neural network-based methods with the corresponding representations. However, these methods tend to focus on individual triples independently, often ignoring neighboring entities that may carry rich structural and semantic information.

In most cases, a knowledge graph can be described as a heterogeneous graph comprising various types of relations [9]. As shown in Fig. 1, the entity *Tom Hanks* is connected to several other entities through different types of relations. The *cast in* relation reflects the movies in which *Tom Hanks* participated, while the *collaborate* relation reflects his collaborators. It is evident that these relations convey different semantic information and exert different influences on the central entity. Therefore, it is inappropriate to treat all relations equally; instead, different attention weights should be assigned selectively to each relation.

* Corresponding author at: Faculty of information and communication engineering, University of Electronic Science and Technology of China, Chengdu 611731, Sichuan, China.

E-mail address: xuj@uestc.edu.cn (J. Xu).

¹ Equally contributed to this work.

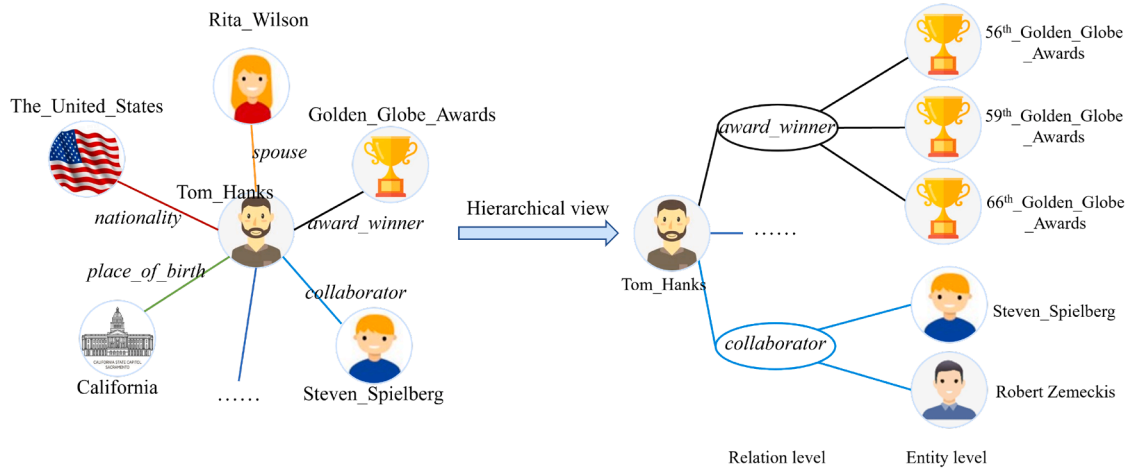


Fig. 1. Example of heterogeneous knowledge graph. The edges with different colors represent different relations. The neighboring entities around Tom_Hanks comprise multiple entities and relations.

The attention mechanism, a cutting-edge paradigm in deep learning [10], enhances a model's capacity to focus on the most salient features [11]. It has demonstrated effectiveness across various domains and is now widely applied in many fields [12]. The Graph Attention Network (GAT) [13], a type of graph neural network, has achieved significant progress in handling graph-structured data. It dynamically assigns varying levels of importance to each node in the neighborhood of a target entity. Despite the success of GAT in modeling graph-structured data, it does not account for multi-relational heterogeneous graphs. Therefore, it is not suitable for direct application in knowledge graph tasks.

Building upon the above analysis, we introduce a novel Hierarchical Graph Attention Network (HGAT) for knowledge graph completion. HGAT employs a hierarchical attention mechanism to capture distinct weights for neighboring entities and relations. Specifically, the initial stage of HGAT consists of the entity-level attention layer, which computes attention scores between a central entity and its neighboring entities based on relation paths. This enables the system to learn how various design parameters or manufacturing conditions in PDMS influence one another. Subsequently, relation-level attention is employed to extract semantic information from distinct relation paths and integrate the features of neighbors associated with those paths. By focusing on these relationships, PDMS can more effectively model the behavior of interconnected systems, such as assembly lines and material supply chains. Leveraging the features extracted through the two-level attention mechanism, our model effectively identifies the optimal combination of neighbors and diverse relation paths in a hierarchical manner. As a result, it captures the rich semantics and intricate structural information embedded in knowledge graphs. Understanding these complex relationships—whether among materials, processes, or design parameters—enables PDMS to support more intelligent, adaptive decision-making that accounts for multilayered interdependencies throughout the product development lifecycle.

Moreover, we propose integrating node importance estimation (NIE) and relation importance estimation (RIE) into the knowledge graph completion model. NIE is a vital area of research in graph networks that focuses on determining the significance or centrality of nodes within a graph based on its structural features. We argue that different entities play different roles within a knowledge graph and that incorporating this information into the model can enhance structural representation and learning. In our work, we estimate node importance using node centrality, specifically by computing the degree of each node. In PDMS, this involves identifying key elements in the manufacturing process—such as critical machines or high-demand materials—that play essential roles in ensuring operational efficiency.

In addition, we introduce relation importance estimation (RIE), which is derived based on estimated node importance. Relation importance is computed by propagating degree information from connected entities to their corresponding relations. This operation enhances interaction modeling between entities and relations, which has been shown to be beneficial in graph representation learning. Our approach allows for a more holistic understanding of the complex interrelationships among components, enabling PDMS to respond to evolving demands with greater agility and precision.

Specifically, the key contributions of our research are summarized as follows:

- We propose a novel hierarchical graph attention network. The hierarchical attention mechanism can simultaneously consider the significance of both entities and relations. It enhances the cognitive capabilities of PDMS, allowing it to better understand and optimize complex design and manufacturing parameters.
- We propose to use node centrality to estimate node importance, which could bring more structural information for model training. It enables the PDMS system to focus more on the key factors in design and manufacturing.
- We estimate the relation importance by passing the degree information from the connected entities to relations. This operation could improve the interactions between the entities and relations and enable a more holistic understanding of the complex relationships between components in PDMS.
- We incorporate node importance estimation (NIE) and relation importance estimation (RIE) techniques. These approaches serve as a versatile and effective module to enhance the representation capability of the graph.

2. Related work

To provide a comprehensive understanding of recent developments in knowledge graph completion (KGC), we first review representative knowledge graph embedding techniques, which lay the foundation by encoding structural and semantic information into continuous vector spaces. Building upon this foundation, we then examine the integration of GNN-based models that aim to more effectively capture relational patterns within graphs. Recognizing the limitations of flat attention mechanisms in modeling complex interactions, we next explore the application of hierarchical attention mechanisms, which offer improved expressiveness and interpretability. Finally, we consider how node and relation importance can be incorporated to better reflect the varying significance of graph components, further enriching the representation learning

Table 1
Comparison of existing models and our proposed model.

Model Category	Representative Models	Key Characteristics	Limitations
Graph Embedding Models	TransE [6], TransD [14], TransF [15], RESCAL [16], DistMult [17], ComplEx [18], ConvE [19], RDF2Vec [20]	Represent relations as translations in embedding space; tensor decomposition; or deep architectures for triples	Ignore local neighborhood context and semantic richness
GNN-based Models	R-GCN [21], SACN/WGCN [22]	Aggregate neighbor information; support multi-relational structure	Treat all neighbors equally; lack fine-grained differentiation of importance
Attention-enhanced Models	HAN [23], MConvKGC [24], DPGAT [25], GATH [31], LTRGN [32]	Introduce attention to highlight informative neighbors or triples	Mostly single-level attention; limited integration of both entity and relation importance
Our Model	Integrates hierarchical attention to distinguish both node-level and relation-level importance; captures heterogeneous significance of neighbors; improves embedding quality by fine-grained importance modeling		

process. Each of these aspects contributes a vital perspective to the ongoing evolution of KGC methodologies. Table 1 summarizes the characteristics and limitations of existing works, as well as the key contributions of this study.

2.1. Knowledge graph embedding

Traditional knowledge graph completion methods typically employ mapping techniques to transform the knowledge graph into a lower-dimensional space. This process preserves both structural and semantic information, thereby enabling downstream tasks to leverage the acquired embeddings. We broadly categorize existing models for knowledge graph completion into three distinct groups:

(i) Translation-based models adopt a learning approach in which relations are modeled as translation operations between head and tail entities. TransE, as the pioneering and one of the most widely used models in this category, operates under the assumption that the combined embedding of $h + r$ should be close to the embedding of t . In addition to TransE [6], numerous variants and extensions have been proposed, including TransD [14], and TransF [15].

(ii) Tensor decomposition models apply tensor factorization techniques to triples, typically expressed as $X_{hrt} \approx h^T M_r t$. Nickel et al. proposed RESCAL [16], which represents each relation using a full-rank matrix and captures rich interactions via tensor products. However, it requires a large number of parameters. DistMult [17] improves on RESCAL by employing a diagonal relation-specific matrix M_r , significantly reducing the parameter space and simplifying training. ComplEx [18] further extends the representation to complex-valued vectors, enabling the modeling of asymmetric relations.

(iii) Deep learning models leverage deep neural networks to encode knowledge graphs. ConvE [19] utilizes 2D convolution to capture interactions between entities and relations. RDF2Vec [20] applies unsupervised language modeling techniques to extract features from RDF graphs by transforming graph structures into word-like sequences.

However, these models often treat triples as independent units, which limits their ability to capture the semantic richness of local neighborhoods. As a result, they may generate lower-quality embeddings.

2.2. GNN For knowledge graph completion

An increasing number of studies have explored the application of graph neural networks (GNNs) to knowledge graph completion (KGC). R-GCN [21] was one of the earliest works to introduce GNNs into KGC tasks and was specifically designed for the multi-relational nature of real-world knowledge bases. It introduces relation-specific transformations to effectively capture the characteristics of multi-relational knowledge graphs. However, R-GCN treats all neighboring entities equally.

SACN [22] proposed the Weighted GCN (WGCN), which divides the graph into subgraphs, each containing only one relation type. WGCN has the ability to dynamically learn the weights for each subgraph during the aggregation of information from neighboring entities.

However, both aforementioned models suffer from the limitation that all neighboring entities associated with a given relation are treated as equally important.

2.3. Hierarchical attention mechanism

Yang et al. [23] introduced a hierarchical attention mechanism consisting of two levels: the lower level includes word encoding and attention, while the upper level incorporates sentence encoding and attention. They demonstrated that this hierarchical model could better reflect the structure of documents, as word-level and sentence-level attentions assign higher weights to more informative content when constructing document representations.

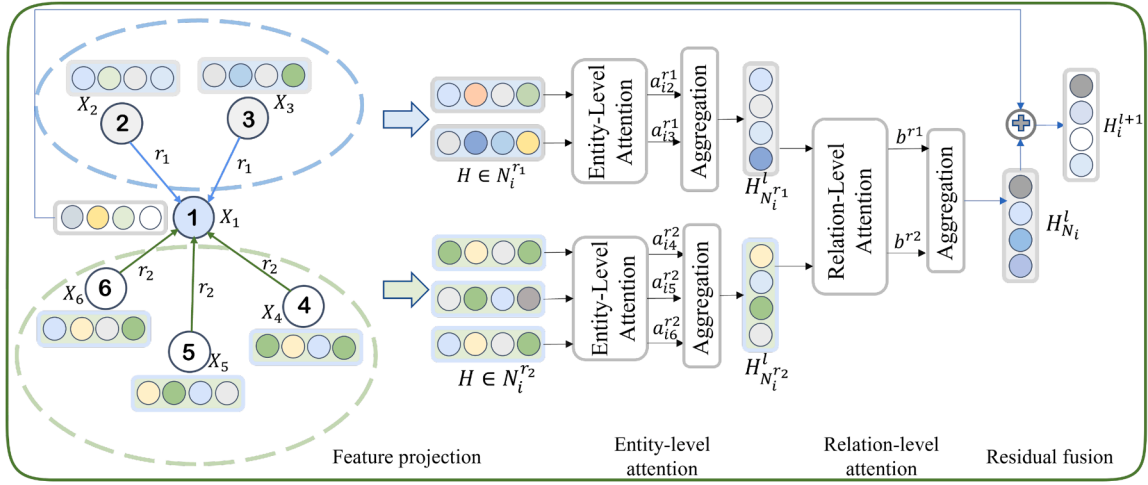
MConvKGC [24] integrates convolutional neural networks with attention mechanisms, where attention is employed to selectively focus on key contextual triples during the learning of local entity and relation features, enhancing the model's ability to capture relational patterns. DPGAT [25] further extends the graph attention structure by incorporating privacy-preserving mechanisms and designing an adaptive attention strategy, which dynamically adjusts attention distributions during training to account for the varying impacts of privacy budgets and learning rates on model performance.

Overall, these approaches demonstrate that attention mechanisms play a crucial role in capturing the heterogeneity and importance of entities and relations within knowledge graphs. Therefore, we propose introducing the hierarchical attention mechanism into knowledge graph-related tasks. The hierarchical attention mechanism can more effectively utilize the neighborhood information of an entity and provide improved interpretability for heterogeneous graph learning.

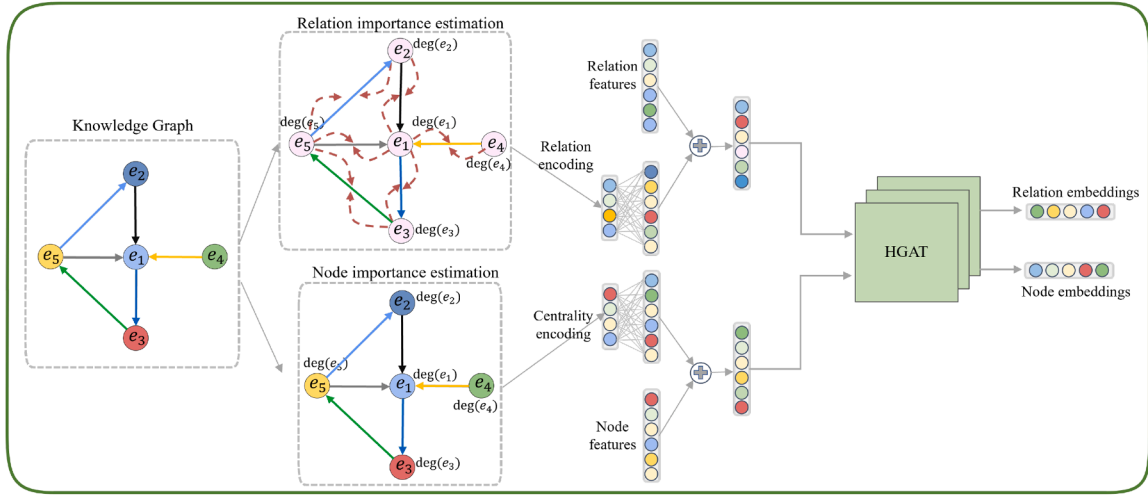
2.4. Node importance and relation importance

Node importance estimation (NIE) is an essential area of research in social networks [26,27] and has found widespread applications in various domains, including recommendation systems [28], the PageRank algorithm [29], and resource allocation [30]. GATH [31] is based on Graph Attention Networks, assigning different attention weights to neighboring nodes to model the importance of adjacent entity features, thereby enabling fine-grained representation of semantic correlations between entities. LTRGN [32] implements an attention mechanism designed to better assess the significance of various neighboring nodes relative to the source node.

In this paper, we propose applying node importance to knowledge graph embedding to enhance the extraction of structural information



(a) Proposed hierarchical graph attention network(HGAT).



(b) Node importance estimation and relation importance estimation.

Fig. 2. (a) Hierarchical graph attention network (HGAT). The initial features are first transformed into high-dimensional embeddings. Then the two-levels hierarchical graph attention network learns the varying attentions of entities and relations. Finally, the residual block fuses the learned embeddings and the initial embeddings to produce the final output. (b) Node importance estimation and relation importance estimation. The dashed red curve represents passing the degree information from entities to connected relations.

for improved graph representation. Additionally, based on the acquired node importance, we introduce relation importance estimation and incorporate relation encoding into our model.

3. Methodology

In this section, we provide a detailed explanation of our proposed methodologies. We begin by presenting the overall framework of the HGAT model. The architecture of the HGAT model is depicted in Fig. 2(a) and incorporates attention mechanisms at both the entity- and relation-level. The entity-level attention aggregates information from neighboring entities based on their relations, while the relation-level attention assigns distinct attention values to different relations. Finally, the residual block fuses the learned embeddings and the initial embeddings and produces the final output. Next, we provide detailed descriptions of node importance estimation and relation importance estimation in Section 3.2. As illustrated in Fig. 2(b), we estimate node and relation importance by extracting the centrality encoding and relation encoding from the graph and applying it to the HGAT model. Finally, the score

function in the decoder part will be presented in Section 3.3. Table 2 shows the relevant notation.

3.1. Hierarchical graph attention network

3.1.1. Entity-level attention

The motivation behind entity-level attention is based on the concept that neighboring entities of different categories may have unique influences on the central entity within a multi-relational graph. For example, when considering the entity *Los Angeles Lakers* and its relation *has players*, it is observed that certain players are more influential than others; hence, it is inappropriate to treat all neighbor features equally for each entity. Therefore, our proposal involves merging neighboring node features based on the same relation-path. The entity-level attention mechanism can aggregate one type of semantic-level features. The inputs of entity-level attention contain a set of entity embedding $X = \{X_1, X_2, \dots, X_N\}$, $X_i \in R^d$, where N is the number of entities and d is the dimension of each entity embedding. To enhance the expressive capability, we incorporate a learnable linear transformation W_0 to modify the input features. W_0 represents a linear transformation matrix that

Table 2
Main notations in the paper.

Notations	Explanations
N	The number of entities
d	The dimension of each entity embedding
W_0	A linear transformation matrix that is parameterized to map the input features into a higher-dimensional feature space
X_i	The feature of the i -th entity
e_{ij}^r	The importance of the j -th neighboring entity relative to relation r
a_{ij}^r	The attention scores of the j -th neighboring entity relative to relation r
$H_{N_i^r}^l$	The hidden features of the r -hop neighbors of node i at layer l
K	the number of attention heads
F_r	Features of all relation path aggregated entities around central entity
b_{r_1}	The attention scores of the relation r_1
deg_i	The degree information of node i
$Z_{deg_i}^+, Z_{deg_i}^-$	The learnable vectors specified by in-degree and out-degree counting, respectively
h_{t_i}, t_{t_i}	The head entity and tail entity of the i -th triplet, respectively

is parameterized to map the input features into a higher-dimensional feature space. Subsequently, the self-attention method is employed to compute the weights between adjacent entities. The procedure can be summarized as follows:

$$e_{ij}^r = \text{attention}(W_0 X_i, W_0 X_j) \quad (1)$$

in which X_i denotes the feature of the central entity, while e_{ij}^r represents the importance of the j -th neighboring entity relative to relation r . Here, attention is any attention function of our choosing used to calculate the similarity between entity i and entity j . In our method, we adhere to the approach employed in GAT and utilize a learnable attention vector a as the attention function. Therefore, the attention function is described as follows:

$$e_{ij}^r = a(W_0 X_i, W_0 X_j). \quad (2)$$

To enable a fair comparison of coefficients across various entities, we utilize the softmax function to normalize them across all possible choices of j :

$$\begin{aligned} a_{ij}^r &= \text{softmax}(e_{ij}^r) \\ &= \frac{e_{ij}^r}{\sum_{k \in N_i^r} \exp(e_{ik}^r)} \\ &= \frac{\exp(\text{LeakyReLU}(a^T [W_0 X_i || W_0 X_j]))}{\sum_{k \in N_i^r} \exp(\text{LeakyReLU}(a^T [W_0 X_i || W_0 X_j]))} \end{aligned} \quad (3)$$

where LeakyReLU is a nonlinear function, $()^T$ represents the transposition operation, and $||$ is the concatenation operation. After obtaining corresponding coefficients of all the neighbor entities, the entity features based on relation-path r could be described as:

$$H_{N_i^r}^l = \frac{1}{|N_i^r|} \sum_{j \in N_i^r} a_{ij}^r H_j^l \quad (4)$$

where l represents l -th layers, $H_{N_i^r}^l$ denotes the aggregated representation of the neighboring entities of node i along the relation path r at the l -th layer. The set N_i^r represents the set of neighbor entities connected to node i via relation path r and $|N_i^r|$ is the cardinality of this set. The coefficient a_{ij}^r reflects the attention weight of the neighbor node j with respect to node i under the context of relation r . H_j^l denotes the feature representation of entity j at the l -th layer.

In order to ensure stability in the learning process and enhance the encapsulation of neighbor information, we follow in GAT and add the multi-head attention mechanism. In particular, K separate attention mechanisms compute the embeddings and are subsequently merged together. The resulting feature representations can be described as follows:

$$H_{N_i^r}^l = \frac{1}{|N_i^r|} \frac{1}{K} \sum_{k=1}^K \sum_{j \in N_i^r} a_{ij}^k H_j^k \quad (5)$$

where K denotes the number of attention heads. Each head computes its own attention coefficients and neighbor representations, capturing different aspects of the local structure.

3.1.2. Relation-level attention

After obtaining the feature embedding under the same relation between entities, we fuse the neighbor feature embedding across different relations. Because different relations may have different feature spaces, directly fusing different types of relation-based neighbor entity features is not suitable. To obtain abundant semantic information from neighbors, it is necessary to selectively aggregate neighbor entity features under different relation-paths. Therefore, we propose to add a relation-level attention that assigns different weights to entity feature embeddings under different relation-paths.

In contrast to the majority of existing methods that solely embed entities within the graph, our model incorporates the learning of a d -dimensional embedding H_r specifically for relations. Specifically, the input of our relation-level attention is H_r^l and $H_{N_i^r}^l$, which could be described as:

$$H_{N_i^r}^l = \delta(H_r^l, H_{N_i^r}^l), r \in R \quad (6)$$

where δ represents any functions to fuse the relation embedding and entities embedding.

Taking inspiration from the channel attention mechanism utilized in computer vision [33] [34], we propose to treat different relational paths around central entities as different channels in a convolutional neural network so that different relations can be assigned with different attention values.

In our paper, we propose to follow the Squeeze and Excitation operation[35] in order to calculate the attention between different relation paths; this operation, shown in Fig. 3, is widely applied to calculate channel attention in the computer vision field. The Squeeze operation can embed the global distribution of relation feature response and aggregate feature maps across spatial dimensions to produce a relation descriptor. This is achieved by using global average pooling and global max pooling to generate relation-wide statistics F_r , which could be described as follows:

$$\begin{aligned} F_r &= \text{Squeeze} \left(H_{N_i^r}^l \right) \\ &= \text{MaxPool} \left(H_{N_i^r}^l \right) + \text{AvgPool} \left(H_{N_i^r}^l \right) \\ &= \max_r \left(H_{N_i^r}^l \right) + \frac{1}{d} \sum_i H_{N_i^r}^l \end{aligned} \quad (7)$$

where d denotes the dimensionality of $H_{N_i^r}^l$ and R signifies the total count of relation types. We can obtain features of all relation path aggregated entities around central entity:

$$F_r = [F_{r_1}, F_{r_1}, \dots, F_{r_R}] \quad (8)$$

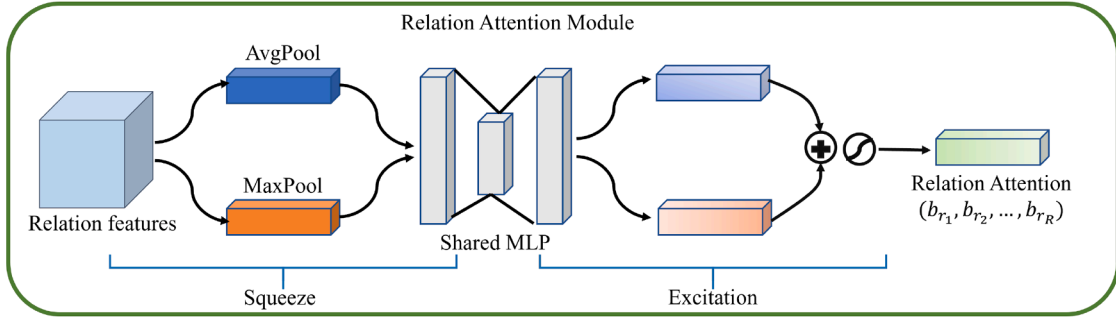


Fig. 3. Relation attention module utilizes both max-pooling outputs and average-pooling outputs with a shared network.

To leverage the aggregated features obtained through the Squeeze operation, the Excitation operation is applied next to capture dependencies between different relation-paths-aggregated features $H_{N_i}^l$ and thereby capture the significance of various relation paths. To achieve this objective, we utilize a straightforward gating mechanism consisting of two fully-connected layers and a *sigmoid* activation function. In short, the relation-path attention is computed as:

$$[b_{r_1}, b_{r_2}, \dots, b_{r_R}] = \sigma(MLP(F_r) + MLP(F_r)) \quad (9)$$

$$= \sigma(W_1 ReLU(W_0(F_{avg}^r)) + W_1 ReLU(W_0(F_{max}^r)))$$

where F_{avg}^r and F_{max}^r represent the average-pooled and max-pooled features over all relation-path-specific embeddings $H_{N_i}^l$, respectively. W_0 and W_1 are the weight matrices of two fully connected layers shared across all relation paths. The use of the *sigmoid* function σ ensures that the computed attention scores $[b_{r_1}, b_{r_2}, \dots, b_{r_R}]$ are bounded between 0 and 1, indicating the relative importance of each relation path.

After the Squeeze and Excitation operation, we finally have all neighbor-aggregated feature $H_{N_i}^l$ around entity i , which can be described as:

$$H_{N_i}^l = \frac{1}{|R|} \sum_{r \in R} (b_r H_{N_i}^l + h_r^l) \quad (10)$$

where R denotes the set of all relation paths, and $|R|$ is its cardinality. b_r is the attention score, which indicates the importance of relation path r , and $H_{N_i}^l$ is the corresponding aggregated feature of neighbors along relation r . The term h_r^l represents a relation-specific bias or residual feature at layer l . By summing over all relation paths and averaging, the final neighbor representation $H_{N_i}^l$ for entity i effectively integrates multi-relation information with learned importance weights.

After obtaining the information from all neighboring entities, we also need to fuse the information of the entity itself H_i^l :

$$H_i^{l+1} = W_2 H_i^l + H_{N_i}^l. \quad (11)$$

However, while learning new embeddings, entities lose their initial embedding information. To address this problem, we employ a linear transformation denoted as W_3 on the initial embedding. Subsequently, we combine the transformed embeddings with the entity embeddings H_i^{l+1} derived from the ultimate attention layer. The equation is:

$$H_i^{l+1} = W_3 H_{init} + H_i^{l+1}. \quad (12)$$

The above operation is to update the entity embeddings. We then update the relation embeddings, which is performed by a nonlinear transformation:

$$H_r^{l+1} = ReLU(H_r^l W_4) \quad (13)$$

After the above update operations, we finally obtain the new entity embeddings H_i^{l+1} and new relation embeddings H_r^{l+1} . In summary, HGAT not only combines the semantic information but also selectively integrates neighbor entity features based on the attention mechanism.

Besides, the hierarchical attention method could provide good interpretability for knowledge graph embeddings, making the results more consistent with human intuition.

3.2. Node and relation importance estimation

We propose to introduce Node Importance Estimation (NIE) into the knowledge graph completion. From a global attention view of the knowledge graph, different nodes may play different roles. For instance, in a Twitter social network, the entity node *Elon_Musk* is considered to be more influential than many other users. Besides, introducing the NIE into the knowledge graph could help the model better extract the structural information, which is beneficial to the feature embedding of entity nodes.

However, node importance information is not directly reflected in the self-attention-based module as it only calculates similarities by using the node semantic features. To address this issue, we propose utilizing node centrality as a means to estimate the importance of nodes in our model. The estimation of node centrality is calculated by counting the degree of the node (Fig. 4(a)). It is a sensible assumption to consider that central nodes with higher degrees exert more influence than those with lower degrees.

In a knowledge graph, it can be explained that the entity with more degrees could reflect that it has more interactions with other entities. Specifically, we add node centrality into our HGAT model by encoding it as a learnable vector. Because centrality encoding is applied to each entity node, we only need to add it into the initial embedding in the input layer. Centrality encoding could be expressed as:

$$H_i^{init} = H_i^{init} + Z_{deg_i}^+ + Z_{deg_i}^- \quad (14)$$

where i is the entity node i , $init$ is the initial feature information, $Z_{deg_i}^+$ and $Z_{deg_i}^-$ are learnable vectors specified by in-degree and out-degree counting, respectively, and deg_i is the degree information of node i .

At the same time, different relations are supposed to have different importance in the knowledge graph. For instance, in an Amazon e-commerce knowledge graph, a user link to the relation *has_hobbies* could better determine the user's shopping propensity than any other relation type. Therefore, we first propose the Relation Importance Estimation (RIE) and provide a relation encoding to our model.

The estimation of the relation importance is based on the node centrality information. We aggregate the degree information of the head and tail entities of each triple to the connected relation. Then we add the relation information of all triples under the same relation together to obtain the encoding of this type of relation. This operation passes the degree information of the entities to the connected relations, which could improve the interactions between the entities and relations. The operation process is demonstrated in Fig. 4(b). The degree information of both the head and tail entities of each triple is aggregated into the features of the corresponding connected relation. Subsequently, the relation information of all triples under the same relation type is aggregated

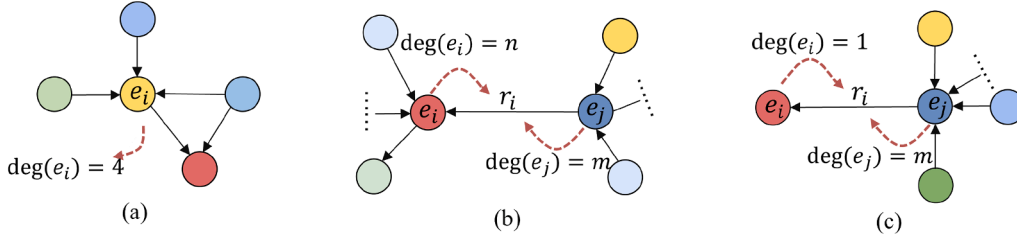


Fig. 4. (a) Calculation of the degree information. (b) Passing the degree information from the entities to connected relations. (c) A misleading situation of the relation importance estimation.

together to form a global importance encoding for that relation category. The transmission of such neighboring entity degree information effectively enhances the feature interaction between entities and relations, facilitating the global propagation of importance information.

However, this computation method may still lead to some misleading cases. As illustrated in Fig. 4(c), when estimating the importance of a relation r_i , one endpoint entity e_i of the relation is merely an isolated node in the graph. In this graph, the primary focus should be on the other node e_j as the relation connecting e_j with the isolated node e_i is not so important. Such situations frequently occur at the edges of graph networks, but the above calculation process cannot distinguish them.

To alleviate this issue, a penalty term is introduced into the formula in this chapter. Specifically, the degree of the tail entity is subtracted by the degree of the head entity and the absolute value of the result is taken. Then, a learnable coefficient A is applied to this absolute value. This penalty term can effectively reduce the weights of edges connected to isolated nodes, thereby improving the accuracy of relation importance measurement. The specific computation formula is given as follows:

$$H_{r_i} = H_{r_i}^{init} + \sum_{t_i \in T_{r_i}} [\deg(h_{t_i}) + \deg(t_{t_i}) - A|\deg(h_{t_i}) - \deg(t_{t_i})|] \quad (15)$$

where T_{r_i} represents the triples based on relation r_i , $init$ means the initial feature information, and $\deg(\cdot)$ is the degree calculation function.

Though centrality encoding and relation encoding are simple, empirical studies show that it can help the model capture semantic information and structural information simultaneously, and improve the performance in the knowledge graph completion task.

3.3. Decoder for link prediction

There are many existing KGC models that can perform the decoder function. We tried many KGC models as a decoder, such as TransE, DistMult, ConvKB, ConvE, and so on. After careful evaluation, we determined that ConvE exhibited the most superior performance, leading us to select ConvE as the decoder in our paper. ConvE leverages a combination of convolutional neural networks and fully connected layers to capture the interactions between input entities and relations. When provided with (h, r, t) triples, ConvE initially converts the embeddings of h and r into two-dimension tensors. It then proceeds to compute the scores of knowledge triplets by utilizing these reshaped tensors. The scoring function employed by ConvE can be represented as follows:

$$f(h, r, t) = \text{ReLU}(\text{vec}(\text{ReLU}(\bar{h}; \bar{r}) * \omega))Q)t. \quad (16)$$

Here, $\bar{h}$ and $\bar{r}$ denote the reshaped versions of h and r as 2D tensors, respectively. The convolution operation is represented by the symbol $*$, the vectorization function is denoted as $\text{vec}()$, and the weight matrix Q corresponds to it.

4. Experiments and discussion

In this section, we begin by performing experiments on the Link Prediction and Entity Classification tasks. Subsequently, we compare the

obtained results with several state-of-the-art methods. We then apply centrality and relation encoding to other classic models for comparative experiments to test their portability. Lastly, we carry out ablation studies and case studies on HGAT to gain insights into the impact of entity-level and relation-level attention on its performance. As no public PDMS-specific knowledge graph is available, we use general datasets with similar structural characteristics for evaluation.

4.1. Experimental settings

4.1.1. Implementation details

The hyperparameters in our models are determined through a grid search conducted during the training phase. We specify the hyperparameter ranges: learning rate $\{0.001, 0.003, 0.005, 0.01\}$; embedding size $\{16, 32, 64, 100, 200\}$; number of attention heads $\{1, 2, 3, 4, 5, 6, 7, 8, 10\}$; dropout rate $\{0.0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6\}$; and batch size $\{8, 16, 32, 64, 128\}$. The optimal hyperparameters are determined by selecting the ones that yield the highest mean reciprocal rank (MRR) on the validation set.

The following parameter values is for all models: batch size of 128, embedding size of 100, and number of attention heads of 6. Specifically, for FB15k-237, the parameters of learning rate, dropout rate are set as 0.001, 0.6, respectively. For WN18RR, the parameters of learning rate, dropout rate are set as 0.003, 0.3, respectively.

Our models are developed using the PyTorch framework and executed on Graphics Processing Units (GPUs) powered by NVIDIA RTX 3090Ti. We use the adaptive moment (Adam) [36] algorithm to train our model. The model contains roughly 7.3 million parameters, with a size of 29.4 MB.

4.1.2. Evaluation protocol

We assess the performance of all models in terms of their ability to infer missing facts in KGs, specifically through the task of relation prediction. We then compute a score for each corrupted score. In particular, the expectation is that positive candidates should come before negative ones. Eventually, the rank of the correct entity is recorded. We present the outcomes of Mean Reciprocal Rank (MRR), Mean Rank (MR), and Hits@n (correctly predicted relations in the top 1, 3, and 10 ranks) as evaluation metrics. Better performance is indicated by higher MRR and Hits@n and lower MR. The metrics equations are as follows:

$$\begin{aligned} MR &= \frac{1}{|Q|} \sum_{q \in Q} q, \\ MRR &= \frac{1}{|Q|} \sum_{q \in Q} \frac{1}{q}, \\ Hits@K &= \frac{|q \in Q : q \leq K|}{Q}, \end{aligned} \quad (17)$$

where Q is the rank obtained from test predictions.

4.2. Performance of HGAT

To verify its performance, we assess our model by conducting the link prediction task and entity classification tasks. Link prediction refers to

Table 3

Performance on link prediction. (C) represents adding the centrality encoding and (R) represents adding the relation encoding. The highest result is in bold.

Models	FB15k-237				WN18RR			
	MRR	Hits@10	Hits@3	Hits@1	MRR	Hits@10	Hits@3	Hits@1
TransE [6]	0.294	0.420	0.284	0.174	0.182	0.444	0.295	0.027
DistMult [17]	0.241	0.419	0.263	0.155	0.430	0.490	0.440	0.390
ComplEx [18]	0.247	0.428	0.275	0.158	0.477	0.510	0.460	0.410
ConvE [37]	0.325	0.501	0.356	0.237	0.430	0.520	0.440	0.400
ConvKB [19]	0.289	0.421	0.371	0.155	0.249	0.524	0.417	0.057
R-GCN [21]	0.164	0.417	0.264	0.151	0.123	0.207	0.137	0.080
SACN [22]	0.350	0.540	0.390	0.260	0.470	0.540	0.480	0.430
CompGCN [38]	0.355	0.535	0.390	0.264	0.479	0.546	0.494	0.443
M2GNN [39]	0.362	0.545	0.398	0.275	0.485	0.562	0.498	0.444
SE-GNN [30]	0.365	0.549	0.399	0.271	0.484	0.572	0.509	0.446
SEPA [40]	0.332	0.509	0.363	0.243	0.481	0.562	0.493	0.441
GATH [31]	0.344	0.527	0.376	0.253	0.463	0.537	0.475	0.426
MConvKGC [24]	0.348	0.535	0.382	0.257	0.466	0.545	0.479	0.428
DTAE [41]	0.360	0.547	0.395	0.269	0.472	0.531	0.486	0.440
LTRGN [32]	0.359	0.541	0.393	0.267	0.481	0.550	0.494	0.446
HGAT	0.357±0.012	0.537±0.009	0.393±0.008	0.267±0.007	0.482±0.013	0.545±0.010	0.498±0.011	0.443±0.008
HGAT(C)	0.360±0.010	0.545±0.008	0.393±0.020	0.268±0.009	0.483±0.007	0.564±0.012	0.501±0.008	0.444±0.011
HGAT(R)	0.367±0.006	0.545±0.011	0.395±0.009	0.271±0.009	0.486±0.010	0.564±0.007	0.504±0.008	0.445±0.008
HGAT(C+R)	0.368±0.008	0.550±0.006	0.398±0.006	0.279±0.004	0.488±0.011	0.566±0.009	0.512±0.010	0.449±0.007

the task of predicting the missing triples in knowledge graph datasets based on the available known triples. On the other hand, entity classification involves inferring labels for entities by considering their features and relations.

4.2.1. Link prediction

4.2.1.1. Datasets. The experiments are based on FB15k-237[42] and WN18RR[43] datasets. WN18RR and FB15k-237 are subsets of WN18 and FB15k. WN18 is derived from WordNet, which is a knowledge graph containing lexical relations between words. FB15k is a subset of the extensively utilized knowledge graph Freebase, encompassing a vast collection of factual information spanning various domains. Recent research findings have revealed that the datasets FB15k and WN18 contain inverse relations. In light of this discovery, FB15k-237 and WN18RR were introduced as modified versions that exclude the reverse relations present in FB15k and WN18.

4.2.1.2. Baselines. We conduct a comparative analysis with the following baseline approaches, which encompass semantic-matching based-methods, translation-based methods, and convolutional-neural-network-based methods.

TransE [6]: TransE models entities and relations as vectors in a d-dimensional space, specifically represented as h , t , and r , correspondingly. It ensures that the embeddings adhere to the translational principle $h + r = t$. TransE is widely recognized as one of the prevalent KGC models.

DistMult [17]: DistMult proposes a bilinear scoring method to evaluate the scores of knowledge triples.

ComplEx [18]: ComplEx extends the DistMult model by representing entities and relations in the complex space, allowing for both symmetric and antisymmetric relations.

ConvKB [19]: ConvKB is another popular convolutional network-based model for link prediction.

ConvE [37]: ConvE first proposes the convolutional network-based model for link prediction.

R-GCN [21]: R-GCN is a variant of graph convolutional networks (GCNs) and specifically designed to handle KGs with multiple relations.

SACN [22]: SACN introduces weighted graph convolutional networks, combining the advantages of both GCN and ConvE.

CompGCN [38]: CompGCN introduces operations for composing entities and relations for each edge in the local neighborhood of a central node, thus extending the capabilities of previous GCN-based models.

M2GNN [39]: M2GNN suggests utilizing a mixed-curvature space with adjustable heterogeneous curvatures and space weights for embedding multi-relational knowledge graph data.

SE-GNN [30]: SE-GNN proposes a semantic evidence aware GNN to help the knowledge representation with better extrapolation performance.

SEPA [40]: SEPA combines the query representations from several models in a unified form in order to incorporate patterns that are independently captured by each model

GATH [31]: GATH incorporates two separate attention network modules that work synergistically to predict the missing entities.

MConvKGC [24]: MConvKGC adopts an asymmetric convolutional block to comprehensively extract the latent interactions from triples and processes the generated feature maps with various attention mechanisms to further learn local dependencies between entities and relations.

DTAE [41]: Employing a parallel strategy, DTAE applies a multidimensional attention mechanism across kernel space.

LTRGN [32]: LTRGN implements an attention mechanism designed to better assess the significance of various neighboring nodes relative to the source node.

4.2.1.3. Results. The experimental results are shown in Table 3. A large number of independent experiments of the HGAT model are conducted, and the results are reported as mean $\pm$ standard deviation in the experimental results tables. From Table 3, we extract the following findings:

(1) Our model significantly outperforms existing link prediction state-of-the-art on FB15k-237[42] and WN18RR [43] datasets. Based on the evaluation of these two datasets using four metrics, our model outperforms fourteen out of fifteen state-of-the-art models in terms of link prediction. This clearly demonstrates the effectiveness of our model.

(2) Table 3 indicates that the highest performance of HGAT is attained when both centrality encoding and relation encoding are applied. Compared with HGAT, HGAT(C+R) makes significant progress: on FB15k-237, the metrics MRR rise by 2.5 %, on Hits@10 by 1.6 %; on WN18RR, MRR increases by 1.4 %, on Hits@10 by 2.9 %.

(3) We find that HGAT(C+R) obtains obvious improvements on the FB15k-237 dataset. We think the reason is because in FB15k-237 there are more than 200 types of relation (Table 5) and abundant triples, which introduces complex feature interactions between entities and relations. Under these circumstances, the NIE and RIE assume a more crucial role in extracting structural information. In contrast, the WN18RR dataset has only 11 types of relations and fewer triples, which leads to less complex feature interactions. As a result, the performance

Table 4
Entity classification accuracy (%).

Models	MUTAG	AM
Feat [45]	77.9	66.7
WL [46]	80.9	87.4
RDF2Vec [20]	67.2	88.3
R-GCN [21]	73.2	89.3
SynGCN [47]	74.8	86.2
WGCN [22]	77.9	90.2
CompGCN [38]	85.3	90.6
RR-GCN-PPV [48]	79.4	84.7
SCENE [49]	75.4	90.1
RHGNN [50]	82.5	90.4
MPERL + R-GCN [51]	80.9	90.4
DPGAT [25]	81.3	-
IBCS [52]	87.3	-
HGAT	87.5±1.3	91.0±0.9

improvement of HGAT(C+R) on WN18RR is not as significant as that on FB15k-237.

4.2.2. Entity classification

4.2.2.1. Datasets. The entity classification task is based on the MUTAG(Node) [20] and AM [44] datasets. The MUTAG is a commonly used dataset for the entity classification task for nitroaromatic compounds and includes 188 samples of chemical compounds with 7 discrete node labels. The AM dataset is from the Amsterdam Museum and describes the relationship between different artifacts, including 11 classes and 133 relations.

4.2.2.2. Baselines. We benchmark our results against the following baselines in entity classification tasks.

Feat[45]: Feat is an automated method for enhancing data by incorporating features derived from Linked Open Data.

WL[46]: WL is a fast feature extraction method that utilizes the Weisfeiler-Lehman test for graph isomorphism as its foundation.

RDF2Vec[20]: RDF2Vec is a methodology that employs language modeling techniques to extract unsupervised features from sequences of words.

SynGCN[47]: SynGCN is a versatile method that leverages graph convolution techniques to learn word embeddings.

RR-GCN-PPV[48]: RR-GCN-PPV leaves all parameters untrained and thus constructs node embeddings by aggregating randomly transformed random representations from neighbors, i.e., with no learned parameters.

SCENE[49]: A methodology to encode diverse traffic scenes in heterogeneous graphs and to reason about these graphs using a heterogeneous Graph Neural Network encoder and task-specific decoders.

RHGNN[50]: RHGNN overcomes the reliance on meta-paths by modeling the heterogeneous graph as a whole instead of splitting it, and jointly obtains personalized feature representations for nodes and edges.

MPERL[51]: This end-to-end network learns the prediction of class affiliation of entities in KGs within a Markov process. The number of computational steps is learned during training using a geometric distribution. At the same time, the loss function combines insights from the field of evidential learning.

DPGAT[25]: Li et al. propose two adaptive optimization strategies to mitigate the distinct effects of learning rate and privacy budget on models: adaptive learning rates and adaptive noise scaling.

IBCS[52]: An Information Bottleneck-constrained denoised Causal Subgraph (IBCS) learning model is developed, which is capable of simultaneously excluding spurious and noisy parts.

4.2.2.3. Results. The test dataset accuracy for the baseline models and our model is presented in Table 4. The results are reported as mean $\pm$ standard deviation in the experimental results tables. Table 4 clearly

Table 5
Datasets statistics summary.

Datasets	Nodes	Relations	Triples in Train/Valid/Test	Average degree
FB15k-237[42]	14,541	237	272115/17535/20466	42.65
Kinship[53]	104	25	8544/1068/1074	205.50
NELL-995[54]	63,917	198	137465/5000/5000	4.60
WN18RR[43]	40,943	11	86835/3034/3134	4.54

demonstrates the superior performance of our proposed HGAT model in entity classification tasks. On the MUTAG dataset, HGAT achieves an accuracy of 87.5%, outperforming all baseline models by a notable margin, including the previous best model IBCS at 87.3%. Similarly, on the AM dataset, HGAT attains an accuracy of 91.0%, surpassing the strongest competitor CompGCN at 90.6%. These improvements underscore the effectiveness of our hierarchical graph attention mechanism in capturing the complex structural and semantic information present in graph data.

The enhanced performance can be attributed to the key innovations in our model design. First, the hierarchical attention mechanism enables the model to simultaneously weigh the importance of entities and relations. The use of node centrality as a metric for estimating node importance enriches the model's structural awareness, allowing it to focus on more influential nodes during training. Additionally, by propagating degree-based information from entities to relations, our model effectively enhances entity-relation interactions, leading to a more comprehensive representation of the underlying graph.

4.2.3. Performance of centrality encoding and relation encoding

To better evaluate the effectiveness and portability of the centrality encoding and relation encoding, we perform a series of comparative experiments on the most advanced models currently available. The findings are presented below.

4.2.3.1. Centrality encoding. In this section, we apply centrality encoding to three categories of KGC models for the link prediction task: (1) Convolutional neural network models including ConvE, (2) Graph convolutional network models including R-GCN and CompGCN, and (3) Graph attention network models (HGAT). The experiments are implemented on four datasets: FB15k-237, Kinship, WN18RR, and NELL-995. We present the part-counterpart results for ConvE in Fig. 5(a), RGCN in Fig. 5(b), CompGCN in Fig. 5(c), and HGAT in Fig. 5(d).

It is obvious that baselines with centrality encoding perform strongly on most datasets, achieving overall best results on NELL-995[54], Kinship[53], and FB15k-237[42]. We can find that our model HGAT(C) achieves obvious improvements on Kinship datasets. This can be explained in that the Kinship dataset is a small and dense graph. To better understand the influence caused by the graph properties, we list the statistical results and count the average degree of these datasets in Table 5. The average degree of Kinship is 205.5, which could provide rich structural information for centrality encoding.

On WN18RR[43], we observe that some baselines with centrality encoding did not significantly improve over existing results. One reason is that WN18RR is sparser than other datasets. It can be found that the WN18RR dataset comprises only 11 relations and the average degree is 4.54. The sparsity of WN18RR makes it hard for centrality encoding to extract sufficient structural information; hence the improvement is limited. Besides, from Fig. 5(b), we can find that RGCN (C) achieves significant improvements over the baseline model, which may be attributed to RGCN performing slightly worse on its own but being more helpful for centrality encoding.

4.2.3.2. Relation encoding. To assess the effectiveness of relation encoding, we perform link prediction experiments on both CompGCN and HGAT models. The results are shown in Table 5, where the top part

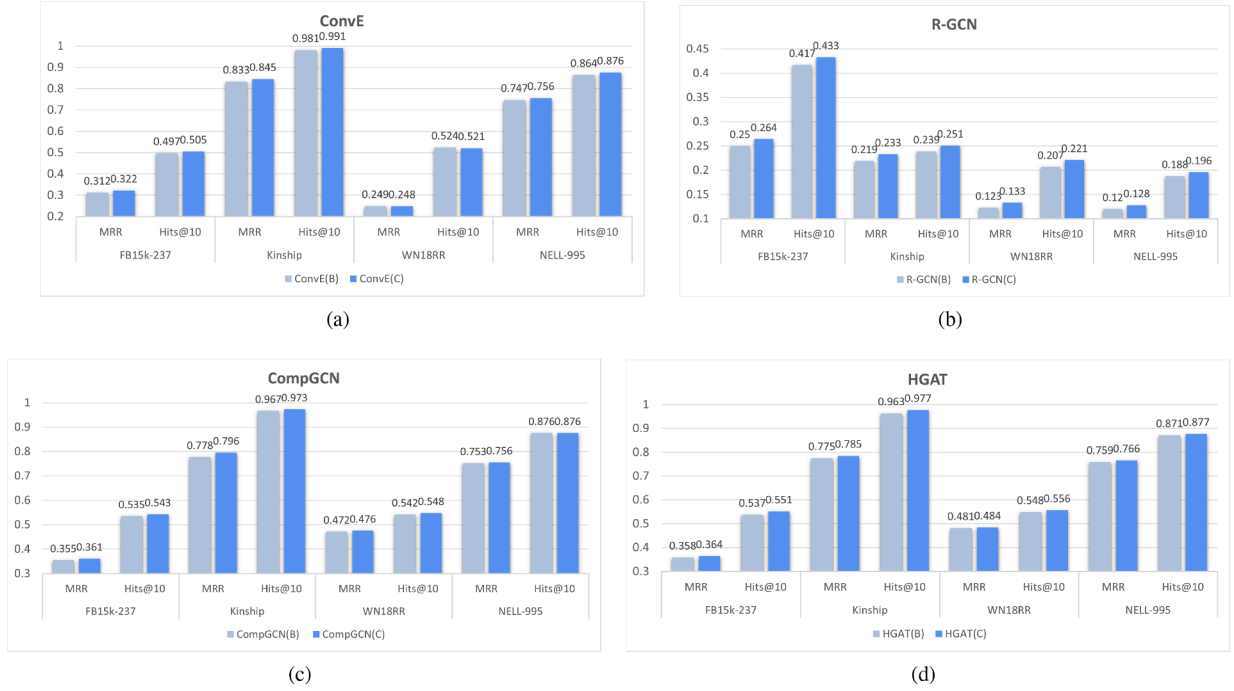


Fig. 5. Performance of centrality encoding on four datasets using (a) ConvE, (b) R-GCN, (c) CompGCN, and (d) HGAT, respectively. (B) represents baseline model, (C) represents baseline model with centrality encoding.

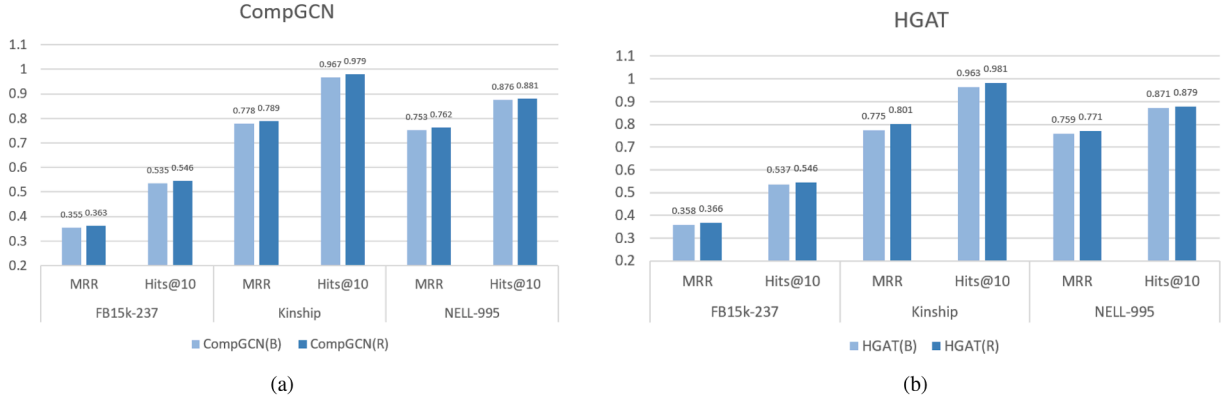


Fig. 6. Performance of relation encoding on three datasets using (a) CompGCN and (b) HGAT, respectively. (B) represents baseline model, (R) represents baseline model with relation encoding.

in each line presents the performance of baselines and the bottom part presents the performance with relation encoding.

From Fig. 6, we can find that relation encoding provides a larger performance boost to the link prediction. Compared with CompGCN(B), CompGCN(R) has significant improvements: on FB15k-237, MRR increases by 0.8%; on Kinship, MRR increases by 0.9%; and on NELL-995, MRR increases by 0.9%. The advances indicate relation encoding could introduce the structural information and increase the interactions between the entities and relations, thereby providing supplementary information for graph learning.

Furthermore, to investigate the effect of the misleading situation in relation importance estimation (Fig. 4(c)), we report the results of relation importance estimation with varying values of A in formula 15. We utilize $A = \{0.1, 0.3, 0.4, 0.6, 0.8, 0.9, 1\}$ for MRR on FB15k-237 and present the results in Fig. 7. From Fig. 7, we observe that the performance of relation encoding generally improves with increasing A . Specifically, it achieves the best performance when $A = 0.9$. The result confirmed our hypothesis about the misleading situation, which could

reduce the effect of a misleading situation via the use of subtraction between head entities and tail entities.

4.3. Case studies

4.3.1. Example analysis of link prediction

Table 6 and Fig. 8 demonstrate an actual link prediction example of hierarchical attention mechanism selected from the FB15k-237 training data. In this case, let's consider the entities *Los Angeles Lakers* and *Tom Hanks* as examples, where a_{ij}^r represents the entity-level attention values and b_{r_i} represents the relation-level attention value.

From Table 6, we can notice that the weights of relations around the entity *Los Angeles Lakers* are different. The relation */sports_team/roster* gets more attention than the relation */sports_team/roster/position*, which means the entities connected with relation */sports_team/roster/position* are not as important as the previous relation. This example aligns with our intuition and serves as evidence that our model effectively discerns the significance of various relations. Moreover, we can find that the weight of neighboring entities around the entity */Tom Hanks* under the

Table 6
Case study for *Los_Angeles_Lakers* and *Tom_Hanks*.

Head entity	Relation	Tail entity
Los_Angeles_Lakers	/sports_team/roster $b_r = 0.314$	Shaquille_O'Neal $a'_{ij} = 0.308$
		Kareem_Abdul-Jabbar $a'_{ij} = 0.089$
		Kobe_Bryant $a'_{ij} = 0.379$
		Center $a'_{ij} = 0.0091$
	/sports_team/roster/position $b_r = 0.0028$	Point_guard $a'_{ij} = 0.0036$
Tom_Hanks	award_honor/award_winner $b_r = 0.218$	Small_forward $a'_{ij} = 0.0014$
		56th_Golden_Globe_Awards $a'_{ij} = 0.145$
		59th_Golden_Globe_Awards $a'_{ij} = 0.172$
		66th_Golden_Globe_Awards $a'_{ij} = 0.131$

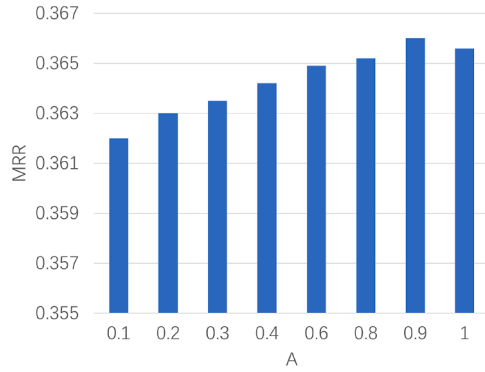


Fig. 7. The Influence of solitary point effect in relationship importance estimation.

relation *award_winner* is different, which demonstrates that entity-level attention was able to assign different weights to neighboring entities.

Based on the above observation, it can be inferred that our hierarchical attention model effectively incorporates the importance of both entities and relations simultaneously. Additionally, the hierarchical attention mechanism showcases its commendable interpretability during the learning process of heterogeneous graphs.

4.3.2. Model evaluation on PDMS scenarios

Our model achieves excellent performance on general benchmark datasets, and it is equally important to further validate its effectiveness in domain-specific PDMS scenarios. Since there is no publicly available PDMS knowledge graph dataset, we constructed a PDMS dataset by extracting structured triples from a patent document related to mechanical design. The resulting knowledge graph captures entities such as components, materials, and assembly relations, reflecting the hierarchical and compositional characteristics of industrial product data. The extracted PDMS knowledge graph contains 284 entities, 15 relation types, and 530 triples, reflecting the hierarchical and compositional characteristics of mechanical product data. Fig. 9 illustrates a partial visualization of the constructed knowledge graph, showing the interconnections among key components and their relationships.

We evaluated our model on this extracted PDMS knowledge graph to assess its capability in handling real-world, domain-specific data. The proposed model achieved a MRR of 0.352 and Hits@10 of 0.527, demonstrating its effectiveness in reasoning over complex entity-relation structures and its adaptability to specialized industrial knowledge.

To provide a more intuitive understanding, we further analyzed several representative reasoning cases. For instance, given the query (*Lamp Housing, includes, ?*), the model correctly inferred entities such as *Lamp Head* and *Lamp Cover*, which are indeed components mentioned in the source patent. This indicates that the model is capable of capturing hierarchical part-whole relationships typical of product design data.

Overall, these results demonstrate that the model not only performs well on general benchmark datasets but also maintains robust perfor-

mance in realistic engineering scenarios. Its capability to infer missing links and recover latent part-whole or material relationships indicates strong potential for deployment in intelligent product design and manufacturing systems.

4.3.3. Parameter sensitivity testing

4.3.3.1. Dimension of embedding d . We first explore the impact of the hidden embedding dimension d on the FB15k-237 and WN18RR datasets. The corresponding results are illustrated in Fig. 10(a) and (c), respectively. As can be seen, the model achieves the best performance when the embedding dimension is set to 100 on both datasets. For smaller dimensions (e.g., $d = 16$), the model lacks sufficient capacity to represent the complex semantic information of entities and relations, resulting in inferior performance. As the dimension increases, the representation power of the embeddings improves, leading to higher MRR scores. However, when d continues to grow beyond 100, the performance begins to decline.

This degradation can be attributed to overfitting and feature redundancy: larger dimensions introduce more parameters, which may capture dataset-specific noise rather than generalizable patterns. In particular, WN18RR shows a more noticeable decline compared to FB15k-237, possibly due to its smaller relation variety and higher sensitivity to noise. Therefore, an embedding dimension around 100 provides a balanced trade-off between expressive capacity and generalization ability.

4.3.3.2. Number of attention heads K . We further analyze the influence of the number of attention heads K in the multi-head attention mechanism, as shown in Fig. 10(b) and (d). Experiments were conducted on both FB15k-237 and WN18RR, varying K within $\{1, 2, 4, 6, 8\}$. The results indicate that the model benefits from using multiple attention heads, with performance steadily improving from one to six heads. The highest MRR is observed when $K = 6$, suggesting that multiple attention heads help the model capture more diverse relational dependencies and improve the robustness of the learned representations.

However, when the number of attention heads exceeds six, the performance begins to deteriorate. This can be explained by attention redundancy, where different heads start to attend to similar patterns rather than providing complementary information. Consequently, increasing K beyond a certain point does not enhance discriminability but instead introduces unnecessary complexity and noise.

Overall, these findings demonstrate that both the embedding dimension d and the number of attention heads K play crucial roles in balancing model expressiveness and generalization. Properly tuning these parameters is essential to achieving stable and optimal performance across different datasets.

4.4. Discussion

4.4.1. Computational complexity analysis

The proposed knowledge graph completion model comprises three main components: an encoder based on a hierarchical graph attention network, a decoder based on two-dimensional randomized reshaping,

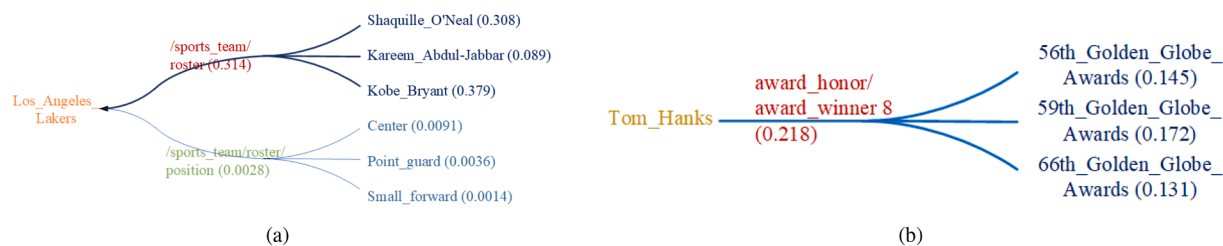


Fig. 8. Case studies: (a) Attention values of neighboring entities and relations around *Los_Angeles_Lakers*. (b) Attention values of neighboring entities and relations around *Tom Hanks*.

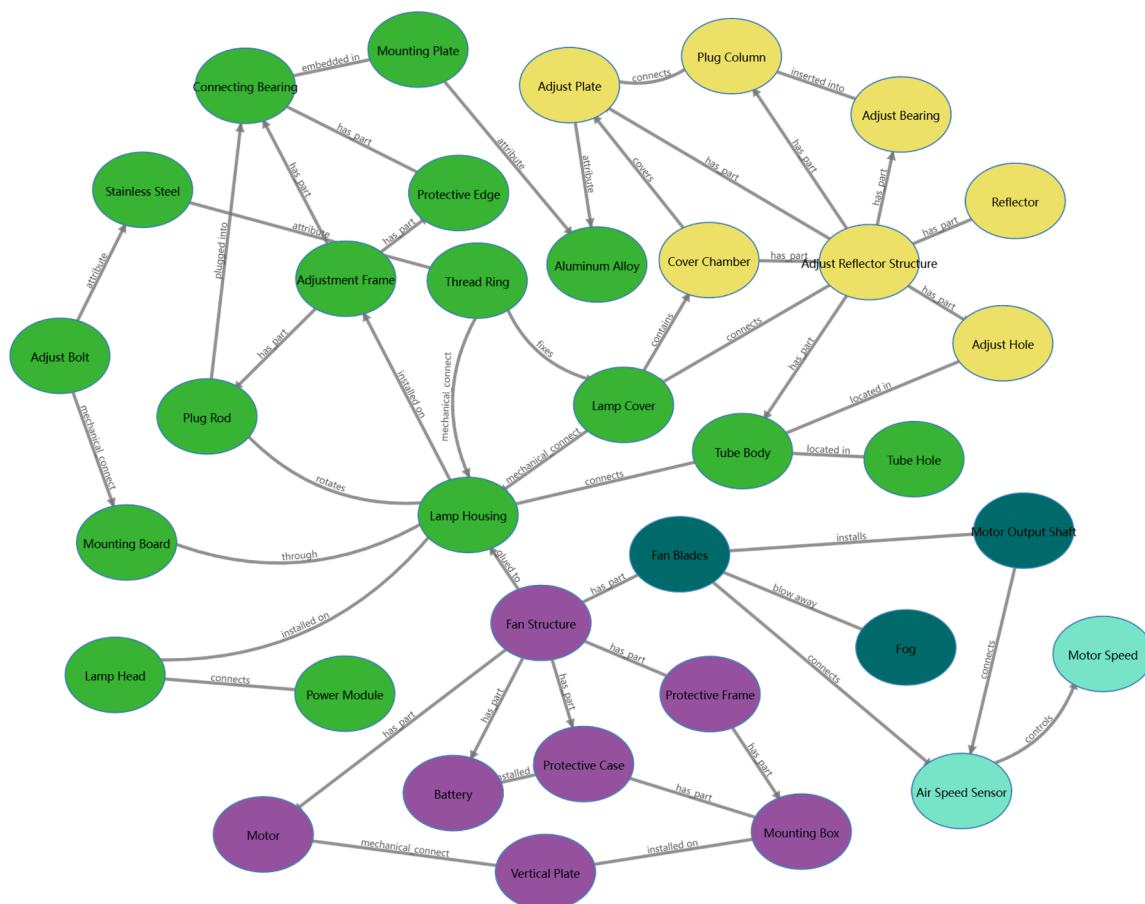


Fig. 9. Visualization of aerial floating mechanical warning light knowledge graph.

and a module for estimating the importance of entities and relations. Accordingly, the computational complexity is analyzed in terms of these three components. Among them, the hierarchical graph attention network is the most computationally intensive. Its complexity is given by $\mathcal{O}((|\mathcal{E}|d^2 + |\mathcal{R}|d)L)$, where d denotes the embedding dimension, $|\mathcal{E}|$ and $|\mathcal{R}|$ are the total numbers of entities and relations in the dataset, respectively, and L is the number of network layers.

Next, the computational complexity of the two-dimensional randomized reshaping decoder is $\mathcal{O}((|\mathcal{E}|d_1 + |\mathcal{R}|d_2)k)$, where d_1 and d_2 represent the embedding dimensions for entities and relations in the decoder, and k denotes the number of reshaping operations.

The entity and relation importance estimation module does not involve neural computations. Instead, it is based on statistical properties of the adjacency matrix; Therefore, its computational overhead has limited impact compared to the first two components.

We further compared the per-epoch training time of our model with several classical GNN-based knowledge graph completion methods. On the FB15k-237 dataset with a batch size of 64, R-GCN requires

186s/epoch and CompGCN takes 269s/epoch, whereas our model requires 308s/epoch. These results demonstrate that our model achieves considerable performance improvements at the cost of a modest increase in computational time, indicating a favorable trade-off between efficiency and effectiveness.

4.4.2. Scalability and real-time applicability

Our computational complexity analysis indicates that the encoder module dominates the training cost, with time and memory complexity proportional to both the number of entities and the embedding dimension. Experimental results demonstrate that HGAT achieves excellent performance and can efficiently handle medium- to large-scale benchmark datasets. However, for ultra-large industrial knowledge graphs, such as those encountered in those scenarios with hundreds of thousands of entities and millions of triples, higher hardware requirements may be necessary. This constitutes a limitation of the current model, as training could become resource-intensive in such scenarios. To mitigate this issue, recent studies—e.g., Li et al.’s distributed training

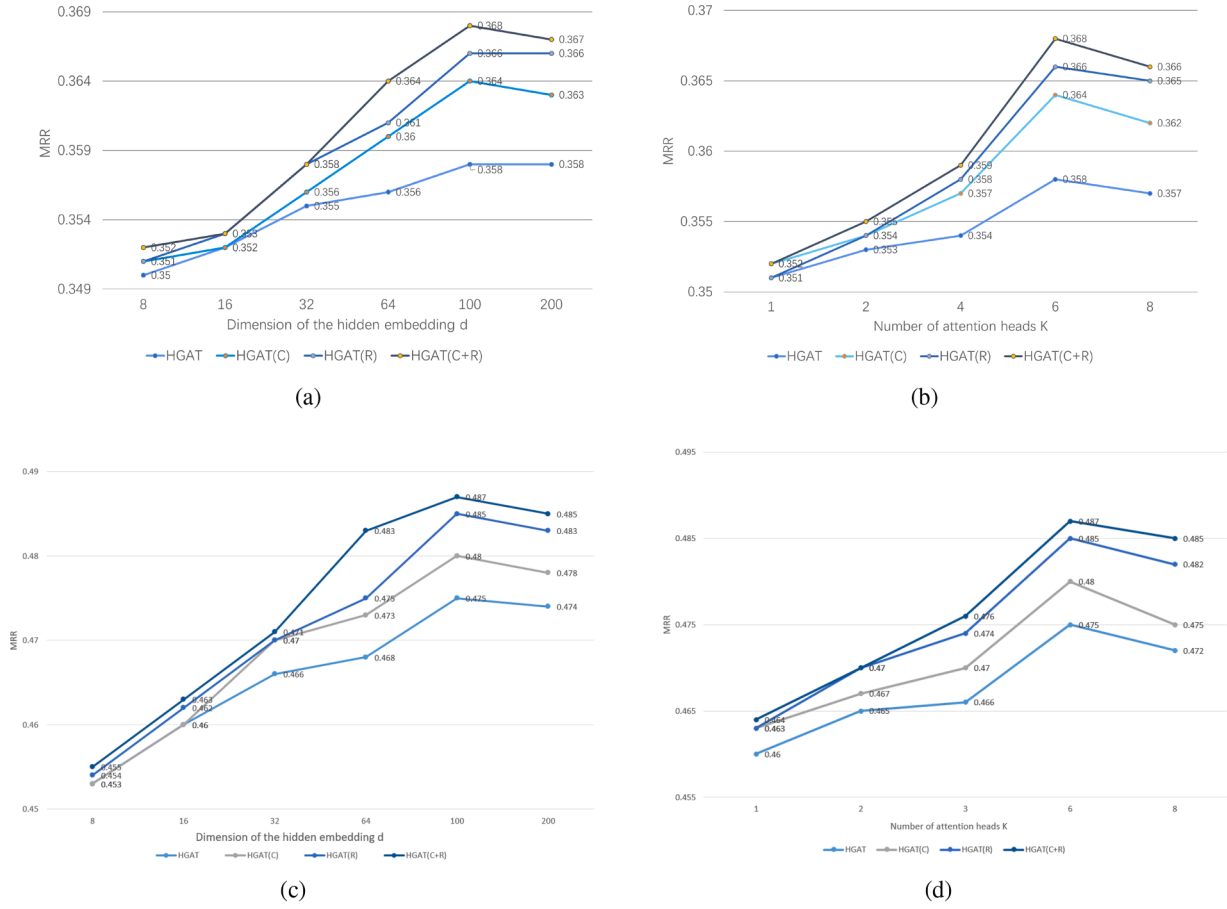


Fig. 10. Parameter sensitivity of (a) dimension d and (b) attention heads K on FB15K-237, and (c) dimension d and (d) attention heads K on WN18RR.

algorithm [55]—demonstrate that distributed training frameworks and device placement strategies can significantly reduce the computational burden. By leveraging such methods, HGAT could scale to industrial-size knowledge graphs while maintaining training efficiency.

In industrial applications, real-time requirements generally focus on inference rather than training. In this context, the decoder and importance estimation modules of HGAT are lightweight and computationally efficient. Once the model is trained, it can perform high-throughput link prediction and entity classification, enabling rapid reasoning over complex hierarchical and compositional relationships among components.

The inference time of our model is 38.71 seconds on FB15k-237, which demonstrates that it can meet the real-time requirements of medium-latency industrial scenarios in PDMS. Specifically, tasks such as design recommendation and component replacement analysis generally tolerate response times ranging from tens of seconds to several minutes. Therefore, our model achieves a reasonable balance between reasoning accuracy and computational efficiency, making it suited for practical PDMS applications.

4.4.3. Integration with other models

While HGAT effectively captures relational dependencies in knowledge graphs, its predictive performance can be further improved by leveraging auxiliary models to provide complementary information.

Nodes and relations may also be associated with textual or visual data. Embeddings extracted from these modalities using neural networks (e.g., BERT [56] for text, VGG [57] for images) can be fused with HGAT embeddings. Concretely, each node embedding can be concatenated or combined via attention-based mechanisms with the corresponding modality-specific embeddings. This multimodal fusion allows HGAT to perform reasoning over enriched node and relation representations, en-

hancing its capability to model complex dependencies in large-scale industrial knowledge graphs.

The main challenges of integrating other modal information with HGAT lie in several aspects. First, features from heterogeneous sources must be properly aligned with the corresponding nodes and relations in the knowledge graph to ensure meaningful fusion. Second, incorporating additional input features inevitably increases computational and memory requirements, which must be carefully managed to maintain efficiency.

Moreover, as discussed in Section 4.4.2, to accommodate industrial-scale large-scale knowledge graphs, distributed training frameworks [55] or a knowledge hypergraph 1-N multilinear scoring approach (Hy-Cube) [58] can be employed to alleviate computational overhead. These techniques enhance the adaptability of HGAT to a wider range of scenarios and improve its practical applicability.

5. Conclusion and future work

In this paper, we propose a hierarchical graph attention network (HGAT) for improving the cognitive ability of product design and manufacturing systems. The HGAT model integrates both entity-level attention and relation-level attention, enabling the selective aggregation of neighboring information pertaining to entities. Through the proposed hierarchical attention method, the significance of neighboring entities and relations surrounding a given entity can be comprehensively taken into account. The hierarchical attention mechanism could also provide good interpretability for a heterogeneous graph. Moreover, centrality encoding and relation encoding are introduced into the heterogeneous graph learning process, which could bring more useful structural information for the graph representation from a global view. Because there

is currently no publicly available knowledge graph dataset of product design and manufacturing systems, we conduct experiments on popular classic knowledge graph datasets. The experiments yield very good results, indicating that our approach can help product design and manufacturing systems achieve a more intelligent and data-driven decision-making process.

Despite the promising performance achieved by our model, several limitations remain to be addressed. While our current approach leverages node degree as a simple yet effective measure of node importance, it is worth exploring more sophisticated centrality metrics in future work. Building on the work of Li et al. [58], measures such as betweenness centrality, or eigenvector centrality may capture more nuanced aspects of node influence within the graph structure. Incorporating these alternative metrics could potentially lead to more expressive importance modeling and further enhance performance.

Additionally, The current model operates under a static graph assumption, whereas some PDMS networks are inherently dynamic and evolve over time. As part of future work, we intend to extend the proposed framework to dynamic graph learning scenarios by integrating temporal dependence.

CRedit authorship contribution statement

Jie Xu: Writing – original draft, Validation, Software, Methodology, Investigation, Conceptualization; **Shumao Zhang:** Writing – original draft, Validation, Software, Methodology, Investigation, Conceptualization; **Haodiao Xie:** Software, Methodology, Conceptualization; **Hailing Zhang:** Validation, Investigation; **Ke Miao:** Software, Methodology; **Qiu-fu Fu:** Investigation, Conceptualization.

Data availability

Data will be made available on request.

Declaration of interests

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This work was supported by the [National Natural Science Foundation of China](#) (Grant No. 62071098) and Sichuan Science and Technology Program (Grant Nos. 2021YFG0307, 2022YFG0319).

References

- [1] J. Li, K. Deng, J. Li, Y. Ren, Session-Oriented fairness-Aware recommendation via dual temporal convolutional networks, *IEEE Trans. Knowl. Data Eng.* 37 (2) (2025) 923–935. <https://doi.org/10.1109/TKDE.2024.3509454>
- [2] Z. Liu, J. Li, Y. Huang, N. Cui, L. Pei, Knowledge-based natural answer generation via effective graph learning, *Knowl. Based Syst.* 316 (2025) 113288. <https://doi.org/10.1016/j.knsys.2025.113288>
- [3] H.J. Levesque, The logic of incomplete knowledge bases, in: *Readings in Artificial Intelligence and Databases*, Elsevier, 1989, pp. 328–341.
- [4] S. Ji, S. Pan, E. Cambria, P. Martinen, S.Y. Philip, A survey on knowledge graphs: representation, acquisition, and applications, *IEEE Trans. Neural Netw. Learn. Syst.* 33 (2) (2021) 494–514.
- [5] A. Rossi, D. Barbosa, D. Firmani, A. Matinata, P. Merialdo, Knowledge graph embedding for link prediction: a comparative analysis, *ACM Trans. Knowl. Discov. Data (TKDD)* 15 (2) (2021) 1–49.
- [6] A. Bordes, N. Usunier, A. Garcia-Duran, J. Weston, O. Yakhnenko, Translating embeddings for modeling multi-relational data, *Adv. Neural Inf. Process. Syst.* 26 (2013).
- [7] Z. Wang, J. Zhang, J. Feng, Z. Chen, Knowledge graph embedding by translating on hyperplanes, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, 28, 2014.
- [8] Y. Lin, Z. Liu, M. Sun, Y. Liu, X. Zhu, Learning entity and relation embeddings for knowledge graph completion, in: *Twenty-ninth AAAI Conference on Artificial Intelligence*, 2015.
- [9] C. Shi, Y. Li, J. Zhang, Y. Sun, S.Y. Philip, A survey of heterogeneous information network analysis, *IEEE Trans. Knowl. Data Eng.* 29 (1) (2016) 17–37.
- [10] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, Ł. Kaiser, I. Polosukhin, Attention is all you need, *Adv. Neural Inf. Process. Syst.* 30 (2017).
- [11] J. Xu, W. Wang, H. Wang, J. Guo, Multi-model ensemble with rich spatial information for object detection, *Pattern Recognit.* 99 (2020) 107098.
- [12] J. Xu, R. Song, H. Wei, J. Guo, Y. Zhou, X. Huang, A fast human action recognition network based on spatio-temporal features, *Neurocomputing* 441 (2021) 350–358.
- [13] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Lio, Y. Bengio, Graph attention networks, (2017). [arXiv:1710.10903](https://arxiv.org/abs/1710.10903)
- [14] G. Ji, S. He, L. Xu, K. Liu, J. Zhao, Knowledge graph embedding via dynamic mapping matrix, in: *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, 2015, pp. 687–696.
- [15] J. Feng, M. Huang, M. Wang, M. Zhou, Y. Hao, X. Zhu, Knowledge graph embedding by flexible translation, in: *Fifteenth International Conference on the Principles of Knowledge Representation and Reasoning*, 2016.
- [16] M. Nickel, V. Tresp, H.-P. Kriegel, A three-way model for collective learning on multi-relational data, in: *ICML*, 2011.
- [17] B. Yang, W.-t. Yih, X. He, J. Gao, L. Deng, Embedding entities and relations for learning and inference in knowledge bases, (2014). [arXiv:1412.6575](https://arxiv.org/abs/1412.6575)
- [18] T. Trouillon, J. Welbl, S. Riedel, É. Gaussier, G. Bouchard, Complex embeddings for simple link prediction, in: *International Conference on Machine Learning, PMLR*, 2016, pp. 2071–2080.
- [19] D.Q. Nguyen, T.D. Nguyen, D.Q. Nguyen, D. Phung, A novel embedding model for knowledge base completion based on convolutional neural network, (2017). [arXiv:1712.02121](https://arxiv.org/abs/1712.02121)
- [20] P. Ristoski, H. Paulheim, Rdf2vec: rdf graph embeddings for data mining, in: *International Semantic Web Conference*, Springer, 2016, pp. 498–514.
- [21] M. Schlichtkrull, T.N. Kipf, P. Bloem, B.R. van den, I. Titov, M. Welling, Modeling relational data with graph convolutional networks, in: *European Semantic Web Conference*, Springer, 2018, pp. 593–607.
- [22] C. Shang, Y. Tang, J. Huang, J. Bi, X. He, B. Zhou, End-to-end structure-aware convolutional networks for knowledge base completion, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, 33, 2019, pp. 3060–3067.
- [23] Z. Yang, D. Yang, C. Dyer, X. He, A. Smola, E. Hovy, Hierarchical attention networks for document classification, in: *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 2016, pp. 1480–1489.
- [24] X. Sun, Q. Chen, M. Hao, Y. Li, B. Sun, Mconvkge: a novel multi-channel convolutional model for knowledge graph completion, *Computing* 106 (3) (2024) 915–937. <https://doi.org/10.1007/s00607-023-01247-w>
- [25] Y. Li, X. Song, K. Gong, S. Liu, W. Li, Differentially private graph neural networks for graph classification and its adaptive optimization, *Expert Syst. Appl.* 263 (2025) 125798. <https://doi.org/10.1016/j.eswa.2024.125798>
- [26] S.P. Borgatti, Centrality and network flow, *Soc. Netw.* 27 (1) (2005) 55–71.
- [27] N. Park, A. Kan, X.L. Dong, T. Zhao, C. Faloutsos, Multiimport: inferring node importance in a knowledge graph from multiple input signals, in: *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2020, pp. 503–512.
- [28] F. Zhang, N.J. Yuan, D. Lian, X. Xie, W.-Y. Ma, Collaborative knowledge base embedding for recommender systems, in: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 353–362.
- [29] L. Page, S. Brin, R. Motwani, T. Winograd, The PageRank citation ranking: Bringing order to the web, Technical Report, Stanford Infolab, 1999.
- [30] R. Li, Y. Cao, Q. Zhu, G. Bi, F. Fang, Y. Liu, Q. Li, How does knowledge graph embedding extrapolate to unseen data: a semantic evidence view, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, 36, 2022, pp. 5781–5791.
- [31] W. Wei, Y. Song, B. Yao, Enhancing heterogeneous knowledge graph completion with a novel GAT-based approach, *ACM Trans. Knowl. Discov. Data* 18 (4) (2024). <https://doi.org/10.1145/3639472>
- [32] W. Liu, B. Qiang, R. Chen, Y. Xie, L. Chen, Z. Chen, Linear self-attention with multi-relational graph for knowledge graph completion: linear self-attention with multi-relational graph for knowledge graph. , *Appl. Intell.* 55 (10) (2025). <https://doi.org/10.1007/s10489-025-06592-1>
- [33] J. Fu, J. Liu, H. Tian, Y. Li, Y. Bao, Z. Fang, H. Lu, Dual attention network for scene segmentation, in: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019, pp. 3146–3154.
- [34] Z. Niu, G. Zhong, H. Yu, A review on the attention mechanism of deep learning, *Neurocomputing* 452 (2021) 48–62.
- [35] J. Hu, L. Shen, G. Sun, Squeeze-and-excitation networks, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 7132–7141.
- [36] D.P. Kingma, J. Ba, Adam: a method for stochastic optimization, (2014). [arXiv:1412.6980](https://arxiv.org/abs/1412.6980)
- [37] T. Dettmers, P. Minervini, P. Stenetorp, S. Riedel, Convolutional 2d knowledge graph embeddings, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, 32, 2018.
- [38] S. Vashishth, S. Sanyal, V. Nitin, P. Talukdar, Composition-based multi-relational graph convolutional networks, (2019). [arXiv:1911.03082](https://arxiv.org/abs/1911.03082)
- [39] S. Wang, X. Wei, C.N. Nogueira dos Santos, Z. Wang, R. Nallapati, A. Arnold, B. Xiang, P.S. Yu, I.F. Cruz, Mixed-curvature multi-relational graph neural network for knowledge graph completion, in: *Proceedings of the Web Conference 2021*, 2021, pp. 1761–1771.
- [40] C. Gregucci, M. Nayyeri, D. Hernández, S. Staab, Link prediction with attention applied on multiple knowledge graph embedding models, in: *Proceedings of the*

- ACM Web Conference 2023, WWW '23, Association for Computing Machinery, New York, NY, USA, 2023, p. 2600–2610. <https://doi.org/10.1145/3543507.3583358>
- [41] W. Deng, Y. Zhang, H. Yu, H. Li, Knowledge graph embedding based on dynamic adaptive atrous convolution and attention mechanism for link prediction, *Inf. Process. Manag.* 61 (3) (2024) 103642. <https://doi.org/10.1016/j.ipm.2024.103642>
- [42] K. Bollacker, C. Evans, P. Paritosh, T. Sturge, J. Taylor, Freebase: a collaboratively created graph database for structuring human knowledge, in: *Proceedings of the 2008 ACM SIGMOD International Conference on Management of Data*, 2008, pp. 1247–1250.
- [43] G.A. Miller, Wordnet: a lexical database for english, *Commun. ACM* 38 (11) (1995) 39–41.
- [44] V.d. Boer, J. Wielemaker, J.v. Gent, M. Hildebrand, A. Isaac, J.V. Ossenbruggen, G. Schreiber, Supporting linked data production for cultural heritage institutes: the amsterdam museum case study, in: *Extended Semantic Web Conference*, Springer, 2012, pp. 733–747.
- [45] H. Paulheim, J. Fümkrantz, Unsupervised generation of data mining features from linked open data, in: *Proceedings of the 2Nd International Conference on Web Intelligence, Mining and Semantics*, 2012, pp. 1–12.
- [46] N. Shervashidze, P. Schweitzer, E.J. Van Leeuwen, K. Mehlhorn, K.M. Borgwardt, Weisfeiler-lehman graph kernels, *J. Mach. Learn. Res.* 12 (9) (2011).
- [47] S. Vashishth, M. Bhandari, P. Yadav, P. Rai, C. Bhattacharyya, P. Talukdar, Incorporating syntactic and semantic information in word embeddings using graph convolutional networks, (2018). [arXiv:1809.04283](https://arxiv.org/abs/1809.04283)
- [48] V. Degraeve, G. Vandewiele, F. Ongenaes, S.V. Hoecke, R-GCN: The r could stand for random, (2022). [arXiv:2203.02424](https://arxiv.org/abs/2203.02424) <https://api.semanticscholar.org/CorpusID:247244996>
- [49] T. Monninger, J. Schmidt, J. Rupprecht, D. Raba, J. Jordan, D. Frank, S. Staab, K. Dietmayer, SCENE: Reasoning about traffic scenes using heterogeneous graph neural networks, *IEEE Rob. Autom. Lett.* 8 (3) (2023) 1531–1538. <https://doi.org/10.1109/LRA.2023.3234771>
- [50] S. Zhu, S. Zhang, Y. Liu, C. Zhou, S. Pan, Z. Li, H. Chen, RHGNN: Imposing relational inductive bias for heterogeneous graph neural network, *Int. J. Mach. Learn. Cybern.* 16 (2) (2024) 855–871. <https://doi.org/10.1007/s13042-024-02305-1>
- [51] J. Mäkelburg, Y. Peng, M. Alam, T. Weller, M. Acosta, Markov process-Based graph convolutional networks for entity classification in knowledge graphs, in: *International Conference on Agents and Artificial Intelligence (ICAART 2025)*, Porto, Portugal, 2025. <https://hal.science/hal-04837417>
- [52] R. Yuan, Y. Tang, Y. Xiao, W. Zhang, IBCS: Learning information bottleneck-Constrained denoised causal subgraph for graph classification, *IEEE Trans. Pattern Anal. Mach. Intell.* 47 (3) (2025) 1627–1643. <https://doi.org/10.1109/TPAMI.2024.3508766>
- [53] S. Kok, P. Domingos, Statistical predicate invention, in: *Proceedings of the 24Th International Conference on Machine Learning*, 2007, pp. 433–440.
- [54] W. Xiong, T. Hoang, W.Y. Wang, DeepPath: a reinforcement learning method for knowledge graph reasoning, (2017). [arXiv:1707.06690](https://arxiv.org/abs/1707.06690)
- [55] A. Long, Y. Wang, X. Huang, W. Han, R. Fan, Y. Chen, J. Li, An efficient device placement method for distributed training of multi-branch neural network-Based remote sensing interpretation, in: M. Barhamgi, H. Wang, X. Wang (Eds.), *Web Information Systems Engineering – WISE 2024*, Springer Nature Singapore, Singapore, 2025, pp. 278–288.
- [56] J. Devlin, M.-W. Chang, K. Lee, K. Toutanova, BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding, 2019. [arXiv:1810.04805](https://arxiv.org/abs/1810.04805)
- [57] K. Simonyan, A. Zisserman, Very Deep Convolutional Networks for Large-Scale Image Recognition, 2015. [arXiv:1409.1556](https://arxiv.org/abs/1409.1556)
- [58] Z. Li, X. Wang, J. Zhao, W. Guo, J. Li, Hycube: efficient knowledge hypergraph 3D circular convolutional embedding, *IEEE Trans. Knowl. Data Eng.* 37 (4) (2025) 1902–1914. <https://doi.org/10.1109/TKDE.2025.3531372>